Learning for Probablistic Intelligence

Learning ProbabilisticModels

- In order to function effectively in uncertain environments, agents need probabilistic models
- These models usually cannot be "pre-programmed" completely and require to be learnt from data collected from the enviroment
- Recall that probabilistic models can be categorised as being *discriminative* (conditional probability models) or *generative* (joint probability models)
- We will look at some simple learning techniques for each of these (no structure-learning, only parameter-learning)

Recall: A Simple Generative Model I

- Let us assume data are are random samples drawn from $(\mathbf{X}, Y)$ or $(X_1, X_2, \ldots, X_d, Y)$
- We know:

$$P(X_1, X_2, \ldots, X_d, Y) \;=\; P(X_1, \ldots, X_d | Y) P(Y)$$

where (from the chain rule):

$$P(X_1, X_2, \ldots, X_d | Y) \;=\; P(X_1 | Y) P(X_2 | X_1, Y) \cdots P(X_d | X_{d-1}, \ldots, X$$

- Maximum likelihood estimates for probabilities are simply *successes*/*total*. A lot of data instances will be needed to get meaningful estimates of any specific $P(x_1, x_2, \ldots, x_d | Y = 1)$ and $(x_1, x_2, \ldots, x_d | Y = 0)$

Recall: A Simple Generative Model II

- Recall the Naive Bayes model, in which we assumed that the X_k are *conditionally independent* of each other, given Y. Then:

$$P(X_1, X_2, \ldots, X_d | Y) \;=\; P(X_1|Y)P(X_2|Y)\cdots P(X_d|Y)$$

  Now, for each value of Y, we only need to estimate $2d$ values, which is huge reduction

- If the estimates are maximum likelihood estimates. That is:

$$P(X_i = x_{ij} | Y = y_k) \;=\; \frac{No.\ of\ instances\ with\ (X_i = x_{ij} \wedge Y = y_k)}{No.\ of\ instances\ with\ Y = y_k}$$

  and:

$$P(Y = y_k) \;=\; \frac{No.\ of\ instances\ with\ Y = y_k}{Total\ Number\ of\ Instances}$$

  (this is an *empirical prior*)

Recall: A Simple Generative Model III

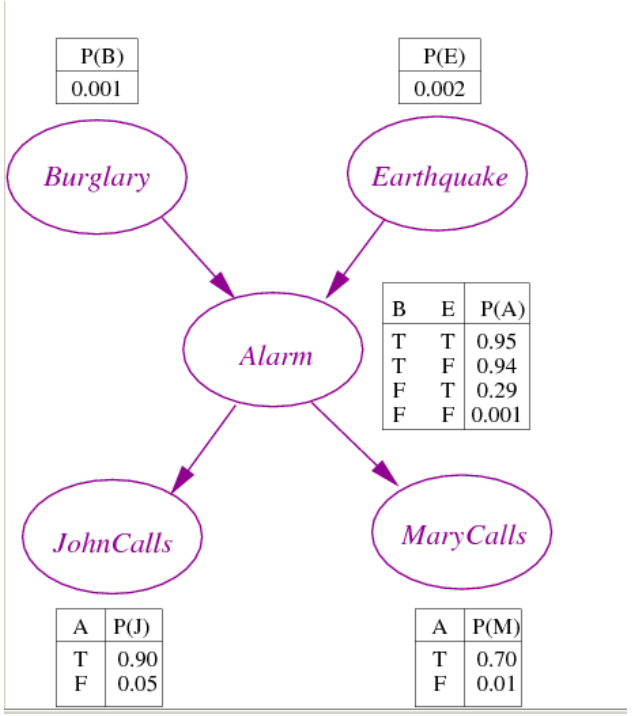
- Once these counts have been obtained from the data, we can compute the values for the joint probability
- In fact, we can also compute the conditional probability $P(Y|X)$
  For example, for the 2-class problem, we will get $P(Y = 1|X) = \alpha \prod_{i=1}^{d} P(X_i|Y = 1)P(Y = 1)$ and $P(Y = 0|X) = \alpha \prod_{i=1}^{d} P(X_i|Y = 0)P(Y = 0) = \beta$. Since these have to add to 1, we can calculate the value of α and the corresponding $P(Y = \ldots|X)$
  - BUT: what happens if the x_{ij} values are continuous? One common assumption is that the class-conditional values x_{ij} are from a Gaussian distribution.

Recall: A Simple Generative Model IV

- That is, for each value of $Y = y_k$, the X_i is a Gaussian defined specific to the X_i and $Y = y_k$. The parameters of the Gaussian are estimated from data using the usual of estimating means and s.d.'s and the corresponding value of of $P(X_i = x_{ij}|Y = y_k)$ is obtained from $P(X_i = x_{ij}|y_k, \mathcal{N}(\hat{\mu}, \hat{\sigma}))$

Learning More Expressive Generative Models I

Recall:



- A Bayesian network is an example of modelling the joint distribution of several observed variables by incorporating prior and conditional dependency information

Learning More Expressive Generative Models II

- "Fitting the (joint) distribution" requires estimation of two aspects of the model: (a) its structure (the DAG); and (b) the CPTs (its parameters)
- The structure is dependent on conditional dependencies in the domain. For the present, we will assume that this known beforehand.

Estimating Parameters in a Bayesian Network I

- Restricting ourselves to Boolean-valued r.v.'s, each entry in the CPT of a BN is like a biased coin. So, estimating the values in a CPT is like estimating the probabilities of many biased coins p_1, p_2, ...
- Recall that the probability p of biased coin landing *heads* can be obtained from data, by modelling them as a sequence of Bernoulli trials. The maximum likelihood estimate (MLE) of p is then:

$$p_{ML} = \frac{s}{n}$$

  where s is the number of *heads* in n trials.
- Problem: MLE is not possible if data are missing (we will need to resort to a generalisation called the Expectation-Maximisation algorithm to estimate values).

Estimating Parameters in a Bayesian Network II

- Another problem: when estimating p if $s = n = 10$ (either real or weighted counts), the MLE is simply 1
  - Does this seem reasonable? (That is, are we now sure that the coin is a double-headed one?)
  - What about if $s = n = 1000$?
- One way is to learn MAP estimates using Bayes Rule

MAP Estimation of a BN's Parameters I

Continuing to assume binary-valued r.v's for the present. Then each CPT entry in the network is simply a specification of coin

	P(X)
X = 1	(a11,b11)

	P(Y=y1|X)
X = 1	(a21,b21)
X =2	(a22,b22)

- For each CPT "coin", we can specify priors using a special distribution called a *Beta* distribution

MAP Estimation of a BN's Parameters II

- This is a distribution that has 2 parameters a, b
- It is a way of specifying a distribution for a r.v. that takes values between 0 and 1 (like the probability of a coin)
- So, for some value of a, b, we $Beta(a, b)$ will give us the probability (actually the *pdf*) to specify our prior knowledge about the probability of heads for a coin
- For example, $Beta(1, 1)$ is a "uniform" prior over values between 0 and 1. $Beta(100, 100)$ is also a uniform prior
- The mean of $Beta(a, b)$ is $\frac{a}{a+b}$
- So, each "coin" i in the CPT will have a prior given by a pair of (a_{ij}, b_{ij}) values representing Beta priors. Bayesian networks with such Beta priors are sometimes called *augmented Bayesian networks*

MAP Estimation of a BN's Parameters III

- The advantage of using *Beta* priors for data that can be represented by "heads" and "tails" is that there is a very simple update rule to get the MAP estimate using Bayes rule
  - If you have a coin with a $Beta(a, b)$ prior and have observed data consisting of s heads and t tails, then the posterior distribution of the probability of heads is $Beta(a + s, b + t)$
  - The MAP estimate of the probability of heads is the mean of the posterior distribution
- For example, suppose you observed data for a network $X \rightarrow Y$ as follows:

S.No.	X	Y
1	x1	y1
2	x1	y1
3	x1	y1
4	x1	y1
5	x1	y1
6	x1	y2
7	x2	y1
8	x2	y1
9	x2	y2
10	x2	y2

MAP Estimation of a BN's Parameters IV

- Then, we update the values for the parameters of the Beta distribution as follows: $a'_{11} = a_{11} + 6$; $b'_{11} = b_{11} + 4$; $a'_{21} = a_{21} + 5$; $b'_{21} = b_{21} + 1$; $a'_{22} = a_{22} + 2$; and $b'_{22} = b_{22} + 2$
  - The probabilities then follow from the expected values of the updated Beta distributions
  - That is $P(X) = \frac{a'_{11}}{a'_{11}+b'_{11}}$; $P(Y = y1|X = x1) = \frac{a'_{21}}{a'_{21}+b'_{21}}$; and so on

Estimating Parameters in BNs for Sequence Data I

- Let us look at constructing a special probabilistic model from data D consisting of a sequence of vectors $\langle \mathbf{x_1}, \mathbf{x_2}, \ldots \mathbf{x_N} \rangle$. For simplicity, we will assume each of the the data instances to consists of values for n Boolean-valued r.v's $X_1, X_2, \ldots X_n$.
- By the chain rule, for any data point $\mathbf{x}$:

$$Pr(\mathbf{X}) = \prod_{i=1}^{d} P(X_i | X_1, X_2, \ldots, X_{i-1}) = \prod_{i=1}^{n} Pr(X_i | \mathbf{X}_{<i})$$

  where $\mathbf{X}_{<i}$ is short for the values of r.v's $X_1, X_2, \ldots X_{i-1}$
- Can you draw the BN DAG for this model?
- Unlike the Naive Bayes model, this kind of model makes no conditional independence assumptions, and is said to be *autoregressive*

Estimating Parameters in BNs for Sequence Data II

- The number of parameters to be estimated is $O(2^n)$ since the last CPT dominates
  - Can we make this more compact, using a functional representation of conditional probabilities (rather than a table)?
- In an autoregressive model the conditionals are specified as functions with a fixed number of parameters

$$Pr(X_i = 1|\mathbf{X}_{<i}) \;=\; Bern(f_i(X_1, X_2, \ldots, X_{i-1}))$$

  where f_i has θ_i parameters. If $X \sim Bern(p)$, then $Pr(X = 1) = p$ and $E[X] = p$. The function f_i is therefore sometimes called the *mean function* for X_i

$$f_i : \{0, 1\}^{i-1} \mapsto [0, 1]$$

- So, the conditional probability for X_i is now restricted to being represented by a function f_i with θ_i parameters

Estimating Parameters in BNs for Sequence Data III

- A simple model:

$$f_i(X_1, \ldots X_{i-1}) \;=\; \sigma(\sum_{k=0}^{i-1} \alpha_k^{(i)} X_k$$

(where $X_0 = 1$), and σ is a sigmoid (S-shaped) function.

  - The logistic function is the sigmoid $\frac{1}{1+e^{-x}}$
  - The parameters θ_i are $\alpha_0^{(i)}, \ldots \alpha_{i-1}^{(i)}$
  - The parameters needed in this autorregressive model is $O(n^2)$ because of the $\sum_1^n i$ terms

More Powerful Autoregression I

- A neural network can be used for a more powerful representation of the mean function f_i
  - NADE: Neural Autoregressive Density Estimation
  - The parameters are now the weights and bias terms in the network and the final sigmoid
  - NADE is a refinement of this approach, with some reduction in parameters to be estimated. The reduction is achieved by sharing parameters (reduces to $O(nd)$, where d layers in each network)

Summary

- Agents often require models that predict probability of events jointly or occurring conditionally.
- For maximum flexibility, it is better for the agent to construct a joint model, since it can then use this model to obtain all the conditional probabilities
- Bayesian Networks are a useful conceptual tool for describing almost any kind of joint probability model
- We looked at some techniques for learning parameters in Bayesian Networks with a known structure.
- Real applications extend these simple techniques with more efficient and powerful technqiues